

Documentation

AR Knick-knack: Project Documentation

Overview & Motivation

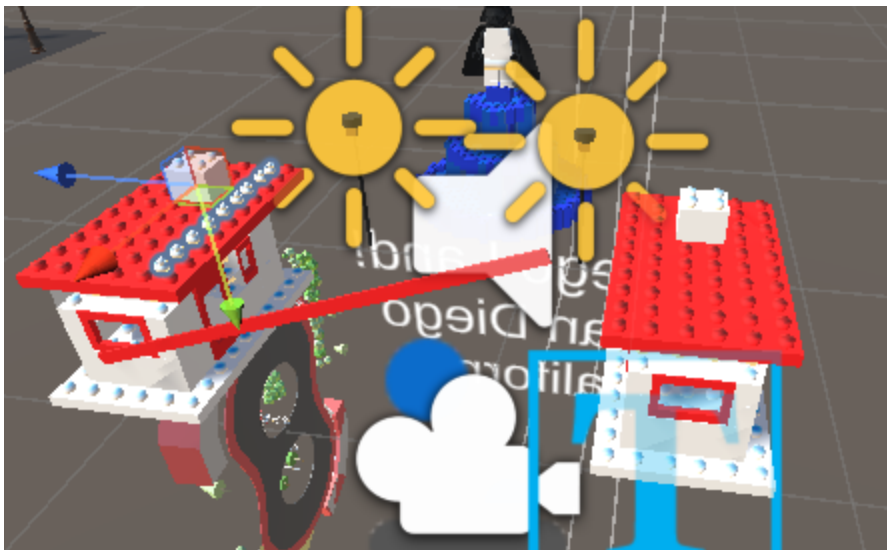
AR Knick-knack is an Augmented Reality application built using Unity and Vuforia. The motivation behind this project was to create an interactive, multi-faceted AR experience that connects physical objects to specific geographic locations and their real-time data.

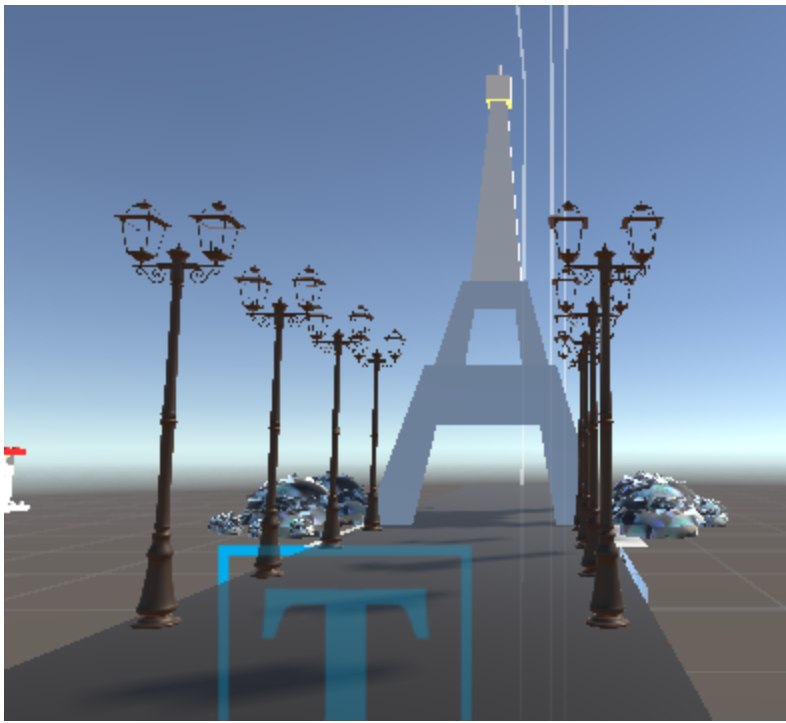
For this project, I chose to highlight **San Diego, California** and **Paris, France**. The application serves as a digital souvenir of those places aka a "knick-knack" that brings apart of my life to the virtual world. I have been to both lego land in San Diego and Paris. I even included an image I took while in paris.

Design & Visual Elements

The core of the application revolves around an AR MultiTarget of a cube. Each side of the cube triggers a different digital augmentation:

- **Top Side** Displays a custom 3D model. This model represents a major attraction in their respective areas. I found the assets for the houses, ducks, and the Lego person. I created the water fountain, the gate, the lighting fixtures, and the scene setup in blender.
- **Sides** around the sides of the cubes are general information of those physical locations in addition you will see the Arc De Triomphe and a go karting loop. These attractions that really stood out to me when I visited these locations.





Development Process

Tools & Libraries

- **Game Engine:** Unity
- **AR SDK:** Vuforia Engine for the cube tracking
- **3D Modeling:** Blender for creating and designing scenes
- **Languages:** C#

Code Structure & Execution

The project is structured around a main Unity scene (`SampleScene`). The Vuforia `MultiTarget` behavior is the parent object, with child `GameObjects` assigned to specific sides (`cube.Top` , `cube.Left` , `cube.Right`).

- Custom C# scripts (`SanDiegoTime.cs` , `SanDiegoClock.cs`) are attached to UI elements to fetch and format the correct time zone data.
- The Vuforia configuration handles the image recognition for the cube's faces.
There are ducks lined up on the roof of one of the entrance barricades. When it's `y` component no longer has a positive direction. They will fly away and return sometime later.

Version Control

I used both github and Unity plasticky for my version control. In github contains basic information about the project. However, I ran into LFS issues early on and decided to move to

using plasticky which I have used once in the past. This is because my desktop doesn't have a webcam and I wanted to ensure that it would run on my mac laptop smoothly.

How to run it:

To run this application, you will need Unity installed with the Vuforia Engine package. Clone the repository, open the project in Unity, print out the MultiTarget cube faces, assemble the physical cube, and press Play in the editor (using a webcam) or build it to a mobile device.

Select the scene in `\scenes\SampleScene` and then run the application.

Challenges & Future Work

Challenges:

One of the primary technical difficulties encountered during this project was managing Unity version control between different operating systems (Mac and Windows desktop). Ensuring that scenes, metafiles, and Vuforia configurations synced correctly without breaking references required careful repository management and troubleshooting. Specifically, ensuring that the child objects stayed parented to the correct MultiTarget faces across platforms took some debugging. Additionally, some of the items weren't snapping to the faces of the cubes correctly. Additionally, having multiple cubes seems to be an issue for my intel mac laptop to capture both of them correctly.

Future Work:

In the future, I would want to make the eiffel tower light up at night as that's something I really enjoyed during my stay there.

AI Usage & Collaboration

AI Usage:

Google Gemini was utilized during the development process primarily for debugging Unity scene configurations and resolving version control conflicts.

Demo Video

https://drive.google.com/file/d/1wjCyKwzODcMZ7q89CN7b1LTgJEf_9Jcb/view?usp=sharing